

Caso de Estudio: Conteo de Palabras con Hadoop MapReduce

Información del Proyecto

Maestría: Ciencia de Datos

Universidad: Pontificia Universidad Javeriana

Materia: Gestión de Datos

Estudiantes: Edwin Silva Salas, Carlos Preciado Cárdenas, Cristian Restrepo Zapata

Fecha de inicio: Diciembre 2025



Reflexión sobre la importancia de Hadoop y Spark en Ciencia de Datos

La gestión y el análisis de grandes volúmenes de datos son desafíos fundamentales en el ámbito de la Ciencia de Datos. Herramientas como **Hadoop** y **Spark** han revolucionado la forma en que abordamos estos retos, permitiendo el procesamiento distribuido y paralelo de datos a una escala que sería inviable con métodos tradicionales.

Hadoop proporciona una infraestructura robusta para el almacenamiento y procesamiento de datos masivos mediante su sistema de archivos distribuido (HDFS) y el paradigma MapReduce. Es especialmente útil cuando se requiere procesar grandes cantidades de información de manera confiable y tolerante a fallos, como en el análisis de logs, procesamiento de textos extensos o integración de datos provenientes de múltiples fuentes.

Por su parte, **Spark** ofrece una plataforma más flexible y veloz para el procesamiento distribuido, permitiendo análisis interactivos, procesamiento en memoria y soporte para tareas avanzadas como machine learning y análisis en tiempo real. Spark es ideal cuando se necesita rapidez en la obtención de resultados, procesamiento iterativo o integración con otras herramientas del ecosistema Big Data.

Utilizaríamos herramientas como Hadoop y Spark en casos como:

- Procesamiento de grandes volúmenes de datos no estructurados (textos, logs, imágenes).
- Análisis de datos históricos para descubrir patrones o tendencias.
- Preparación y limpieza de datos a escala antes de aplicar modelos de machine learning.
- Procesamiento en tiempo real de flujos de datos (Spark Streaming).
- Integración y análisis de datos provenientes de diferentes sistemas y fuentes.

Hadoop y Spark son pilares en la Ciencia de Datos moderna, habilitando el análisis eficiente y escalable de datos masivos, y permitiendo a los científicos de datos extraer valor y conocimiento de información que, de otro modo, sería inmanejable.

Introducción

El análisis de frecuencia de palabras es una técnica fundamental en el procesamiento de lenguaje natural y minería de textos, utilizada para identificar patrones, extraer información relevante y comprender la estructura de documentos textuales. En el contexto de Big Data, donde los volúmenes de información

crecen exponencialmente, se requieren herramientas capaces de procesar grandes cantidades de datos de manera eficiente y escalable.

Este trabajo presenta la implementación técnica de un sistema de conteo y análisis de frecuencia de palabras aplicado a un documento académico sobre inteligencia artificial en el sector bancario. La solución desarrollada integra múltiples tecnologías: extracción de texto desde formato PDF, almacenamiento en HDFS (Hadoop Distributed File System).

Entorno de Trabajo

- **Sistema Operativo:** Ubuntu 22.04 LTS en WSL (instalado con WSL)
- **Hadoop:** Versión 3.3.6
- **Python:** Versión 3.x
- **Documento Origen:** Inteligencia_Rodriguez_ICE_2022.pdf

Paso 1: Conversión de PDF a Texto Plano

Descripción

El primer paso del proceso consiste en extraer el contenido textual del documento PDF académico "Inteligencia Artificial en el Sector Bancario" y guardarlo en un archivo de texto plano que pueda ser procesado por Hadoop.

Herramienta Utilizada

Se utiliza la herramienta **pdftotext** del paquete **poppler-utils**, que permite extraer texto de archivos PDF manteniendo la estructura del contenido.

Implementación

Se implementa archivo notebook (.ipynb) **conteo_palabras_hadoop** para ir ejecutando cada proceso e implementacion de Hadoop en en el conteo de palabras.

De esta forma, el notebook documenta y automatiza cada paso, permitiendo ejecutar el proceso completo de principio a fin desde un solo archivo interactivo. Actualmente, solo los scripts **mapper.py** y **reducer.py** permanecen como archivos independientes, ya que son requeridos por Hadoop Streaming para el procesamiento distribuido.

A continuacion el codigo utilizado para la conversión del archivo .pdf a formato .txt con todas sus respecivas filas en el orden en que se encontraban en el archivo original, utilizando la librerias **subprocesos** que permite utilizar librerias y funciones propias del sistema operativo **Linux** y para este trabajo del sistema operativo **Ubuntu**.

```
import subprocess
import os

# Rutas de archivos
PDF_FILE = "/home/esilvas/Documents/hadoop-
python/files/Inteligencia_Rodriguez_ICE_2022.pdf"
OUTPUT_FILE = "/home/esilvas/Documents/hadoop-
```

```
python/files/texto_plano.txt"

try:
    print(f"Convirtiendo PDF a texto plano...")
    print(f"Archivo origen: {PDF_FILE}")
    print(f"Archivo destino: {OUTPUT_FILE}")

    # Ejecutar pdftotext para extraer el texto
    subprocess.run([
        'pdftotext',
        PDF_FILE,
        OUTPUT_FILE
    ], check=True)

    # Verificar que se creó el archivo
    if os.path.exists(OUTPUT_FILE):
        # Obtener tamaño del archivo
        size = os.path.getsize(OUTPUT_FILE)
        print(f"\nConversión exitosa!")
        print(f"Archivo creado: {OUTPUT_FILE}")
        print(f"Tamaño: {size} bytes")

        # Contar líneas y palabras
        with open(OUTPUT_FILE, 'r', encoding='utf-8') as f:
            lines = f.readlines()
            words = sum(len(line.split()) for line in lines)

        print(f"Líneas: {len(lines)}")
        print(f"Palabras aproximadas: {words}")
    else:
        print("Error: No se pudo crear el archivo de salida")

except subprocess.CalledProcessError as e:
    print(f"Error al ejecutar pdftotext: {e}")
except Exception as e:
    print(f"Error: {e}")
```

Resultado de la Conversión

```
Convirtiendo PDF a texto plano...
Archivo origen: /home/esilvas/Documents/hadoop-
python/files/Inteligencia_Rodriguez_ICE_2022.pdf
Archivo destino: /home/esilvas/Documents/hadoop-
python/files/texto_plano.txt

Conversión exitosa!
Archivo creado: /home/esilvas/Documents/hadoop-
python/files/texto_plano.txt
Tamaño: 66525 bytes
```

Líneas: 1116
Palabras aproximadas: 9574

Estructura del Archivo Resultante

El archivo `texto_plano.txt` generado contiene:

- **Tamaño:** 66,525 bytes (≈ 65 KB)
- **Líneas:** 1,116
- **Palabras:** Aproximadamente 9,574

Muestra del Contenido (Primeras líneas):

Teresa Rodríguez de las Heras Ballell*

INTELIGENCIA ARTIFICIAL EN EL SECTOR
BANCARIO: REFLEXIONES SOBRE SU RÉGIMEN
JURÍDICO EN LA UNIÓN EUROPEA

El objetivo de este trabajo es aproximarnos al estado actual del régimen jurídico aplicable en la Unión Europea al uso de sistemas de inteligencia artificial en el sector financiero, y reflexionar sobre la necesidad de formular principios y reglas que aseguren una automatización responsable de los procesos de toma de decisiones y que sirvan de guía para implementar soluciones de inteligencia artificial en la actividad bancaria.

Paso 2: Carga de Datos a HDFS

2.1 Descripción

En este paso, el archivo de texto plano extraído del PDF (`texto_plano.txt`) se carga en el sistema de archivos distribuido de Hadoop (HDFS). Esto permite que los datos estén disponibles para ser procesados de manera distribuida por los nodos de Hadoop.

2.2 Comandos utilizados

```
# Crear el directorio de entrada en HDFS (si no existe)
hdfs dfs -mkdir -p /user/esilvas/wordcount/input

# Subir el archivo de texto a HDFS
hdfs dfs -put /home/esilvas/Documents/hadoop-python/files/texto_plano.txt
/user/esilvas/wordcount/input/
```

```
# Verificar que el archivo está en HDFS
hdfs dfs -ls /user/esilvas/wordcount/input/
```

2.3 Importancia

Cargar los datos a HDFS es fundamental para aprovechar el procesamiento distribuido de Hadoop. HDFS permite que los datos sean accesibles por todos los nodos del clúster, facilitando la ejecución eficiente de tareas MapReduce sobre grandes volúmenes de información.

2.4 Automatización con Python

La automatización de la carga de datos a HDFS, así como la verificación y gestión de archivos, también se encuentra integrada en el notebook [conteo_palabras_hadoop.ipynb](#).

Paso 3: Implementación MapReduce

3.1 Descripción

En este paso se implementa el modelo MapReduce para el conteo de palabras usando Python. Se desarrollan dos scripts: [mapper.py](#) y [reducer.py](#), que serán utilizados por Hadoop Streaming para procesar el archivo de texto cargado en HDFS.

3.2 Código del Mapper

El Mapper lee cada línea del archivo de entrada, la divide en palabras y emite cada palabra junto con el valor 1:

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print(f"{word}\t1")
```

3.3 Código del Reducer

El Reducer recibe las salidas del Mapper agrupadas por palabra, suma los valores y emite el total de ocurrencias de cada palabra:

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
```

```
word, count = line.split('\t', 1)
try:
    count = int(count)
except ValueError:
    continue
if current_word == word:
    current_count += count
else:
    if current_word:
        print(f"{current_word}\t{current_count}")
    current_word = word
    current_count = count
if current_word == word:
    print(f"{current_word}\t{current_count}")
```

Ambos scripts se guardan en la carpeta `/code/` y se les otorgan permisos de ejecución.

Nota: Los scripts `mapper.py` y `reducer.py` están disponibles en el repositorio de GitHub y deben descargarse desde allí para su uso con Hadoop Streaming.

Paso 4: Ejecución y Resultados

4.1 Ejecución del trabajo MapReduce con Hadoop Streaming

Se ejecuta el trabajo MapReduce usando Hadoop Streaming, especificando los scripts de Mapper y Reducer creados anteriormente. El archivo de entrada es el texto cargado en HDFS y la salida será un archivo de texto con el conteo de palabras.

Comando utilizado:

```
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-input /user/esilvas/wordcount/input/texto_plano.txt \
-output /user/esilvas/wordcount/output \
-file mapper.py \
-file reducer.py \
-mapper "./mapper.py" \
-reducer "./reducer.py"
```

Descarga y visualización del resultado

Para descargar el archivo de salida de HDFS a la máquina local:

```
hdfs dfs -cat /user/esilvas/wordcount/output/part-* >
./files/resultado_conteo_palabras.txt
```

4.2 Muestra de los resultados obtenidos

Las primeras líneas del archivo de resultados son:

```
%      5
&      10
( ... )» . 1
(1.a    1
(20)–    1
(2010) . 1
(2015) . 2
(2015,   1
(2016) . 4
(2017) . 7
(2017,   1
(2018)   1
(2018),  1
(2018) . 5
(2018,   1
(2019),  1
(2019) . 2
(2019,   1
(2019a) . 1
(2019a,  1
(2019b) . 1
(2020) . 2
(2020/2014(INL)) . 1
(2020a) . 2
(2020b) . 2
(2020c) . 1
(2020d) . 1
(2020e) . 1
(2021),  1
(2021) . 4
```

El [archivo](#) completo contiene el conteo de todas las palabras y símbolos presentes en el documento procesado.

Conclusiones

De acuerdo con lo revisado sí, los resultados obtenidos validan empíricamente la Ley de Zipf aplicada al idioma español.

Al observar el output del código, notamos que las palabras con mayor frecuencia absoluta (Top 5) son:

de (815 repeticiones) la (367 repeticiones) y (248 repeticiones) en (234 repeticiones) el (218 repeticiones)

Esto confirma que el la mayor cantidad de datos en un texto en español está compuesto por "Stop Words" (artículos, preposiciones y conjunciones) que aportan estructura gramatical pero no significado semántico por sí solas, es decir no darán un norte completo del panorama que se quiera revisar en el texto.

Sin embargo, el análisis se vuelve valioso al filtrar estas palabras funcionales. Inmediatamente después, emergen los términos que definen el tópico del documento: "ia" (135), "sistemas" (74), "sector" (53) y "ley" (38). Esto demuestra que la herramienta fue capaz de "leer" la temática del documento (Regulación de IA en el sector bancario) a través de la frecuencia estadística, separando correctamente el ruido gramatical de la señal informativa.

El proceso de conteo de palabras usando Hadoop MapReduce permitió automatizar el análisis de frecuencia de términos en un documento académico extenso. Se demostró la integración de herramientas de extracción de texto, almacenamiento distribuido y procesamiento paralelo, logrando:

- Extraer texto de un PDF de manera automatizada.
- Cargar y gestionar datos en HDFS.
- Implementar y ejecutar un flujo MapReduce real con scripts Python personalizados.
- Obtener resultados exportables y analizables en formato tabular.

Este flujo es escalable y puede adaptarse a volúmenes de datos mucho mayores, mostrando la potencia de Hadoop para tareas de procesamiento masivo de texto.

Para mayor informacion sobre la instalacion de Ubuntu y Hadoop en el entorno WSL de Windows, consultar el siguiente documento: [Informe_Instalacion_Hadoop.pdf](#)

Repositorio GIT

- https://github.com/esilvas1/conteo_de_palabras_MCD.git

Referencias

- Apache Hadoop Documentation: <https://hadoop.apache.org/docs/r3.3.6/>
- Poppler Utils: <https://poppler.freedesktop.org/>
- Python Subprocess Documentation: <https://docs.python.org/3/library/subprocess.html>